AI ASSISTED CODING

ASSIGNMENT-12.5

Name: Hari Priya

H.T.No:2303A51104

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order. o Ask AI to include time complexity and space complexity in the function docstring. o Verify the generated code with test cases.

Expected Output: o A functional Python script implementing Merge Sort with proper documentation.

```python
#Generate a function merge_sort that takes a list as input and returns a sorted list
# using the merge sort algorithm also include time and space complexities in docstring and give 2 example testcases
def merge_sort(arr):
    """
    Sorts a list using the merge sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A sorted version of the input list.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list."""
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
```

```python
    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example test cases
if __name__ == "__main__":
    # Test case 1
    arr1 = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", arr1)
    print("Sorted array:", merge_sort(arr1))

    # Test case 2
    arr2 = [5, 2, 9, 1, 5, 6]
    print("\nOriginal array:", arr2)
    print("Sorted array:", merge_sort(arr2))
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_12.5.py
Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]

Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found. o Include docstrings explaining best, average, and worst-case complexities. o Test with various inputs.

• Expected Output: o Python code implementing binary search with AI-generated comments and docstrings.

```python
#Generate a function for binary search that searches for a target value in a sorted list and returns the index of the target value if found,
# otherwise returns -1.Include average,best and worst case time complexities in docstring and give 2 example testcases
def binary_search(arr, target):
    """
    Searches for a target value in a sorted list using binary search algorithm.

    Time Complexity:
    - Average Case: O(log n) - where n is the number of elements in the list.
    - Best Case: O(1) - when the target value is at the middle of the list.
    - Worst Case: O(log n) - when the target value is not present in the list.

    Parameters:
    arr (list): A sorted list to search through.
    target: The value to search for.

    Returns:
    int: The index of the target value if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the list
    return -1
```

```python
# Example test cases
if __name__ == "__main__":
    # Test case 1
    arr1 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    target1 = 5
    print("Array:", arr1)
    print("Target:", target1)
    print("Index of target:", binary_search(arr1, target1))

    # Test case 2
    arr2 = [10, 20, 30, 40, 50]
    target2 = 25
    print("Array:", arr2)
    print("Target:", target2)
    print("Index of target:", binary_search(arr2, target2))
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_12.5.py
Array: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Target: 5
Index of target: 4
Array: [10, 20, 30, 40, 50]
Target: 25
Index of target: -1
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

```python
#A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee.
# The system needs To:
#1. Search appointments using appointment ID.
#2. Sort appointments based on time or consultation fee.
def search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by its ID.

    Parameters:
    appointments (list): A list of appointment records, where each record is a dictionary containing appointment details.
    appointment_id: The ID of the appointment to search for.

    Returns:
    dict: the appointment record if found, otherwise None.
    """
    for appointment in appointments:
        if appointment['appointment_id'] == appointment_id:
            return appointment
    return None
def sort_appointments(appointments, key):
    """Sorts appointments based on a specified key (either 'time' or 'fee').

    Parameters:
    appointments (list): A list of appointment records, where each record is a dictionary containing appointment details.
    key (str): The key to sort by ('time' or 'fee').
    Returns:
    list: A sorted list of appointment records based on the specified key.
    """
    if key == 'time':
        return sorted(appointments, key=lambda x: x['appointment_time'])
    elif key == 'fee':
        return sorted(appointments, key=lambda x: x['consultation_fee'])
    else:
        raise ValueError("Invalid key. Use 'time' or 'fee'.")
```

```python
# Example usage
if __name__ == "__main__":
    appointments = [
        {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100},
        {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150},
        {'appointment_id': 'A003', 'patient_name': 'Alice Johnson', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 09:30', 'consultation_fee': 120},
    ]

    # Search for an appointment
    appointment_id = 'A002'
    result = search_appointment(appointments, appointment_id)
    print(f"Search result for appointment ID {appointment_id}: {result}")

    # Sort appointments by time
    sorted_by_time = sort_appointments(appointments, key='time')
    print("Appointments sorted by time:")
    for appointment in sorted_by_time:
        print(appointment)

    # Sort appointments by consultation fee
    sorted_by_fee = sort_appointments(appointments, key='fee')
    print("Appointments sorted by consultation fee:")
    for appointment in sorted_by_fee:
        print(appointment)
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python310\python.exe c:/Users/gudah/OneDrive/Documents/AI/AC/Lab_assignment_12.5.py
Search result for appointment ID A002: {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}
Appointments sorted by time:
{'appointment_id': 'A003', 'patient_name': 'Alice Johnson', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 09:30', 'consultation_fee': 120}
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
{'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}
Appointments sorted by consultation fee:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
{'appointment_id': 'A003', 'patient_name': 'Alice Johnson', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 09:30', 'consultation_fee': 120}
{'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #4: Railway Ticket Reservation System Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python.

```python
#A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The
#system must:
#1. Search tickets using ticket ID.
#2. Sort bookings based on travel date or seat number.
def search_ticket(bookings, ticket_id):
    """
    Searches for a ticket by its ID.

    Parameters:
    bookings (list): A list of booking records, where each record is a dictionary containing booking details.
    ticket_id: The ID of the ticket to search for.

    Returns:
    dict: The booking record if found, otherwise None.
    """

    for booking in bookings:
        if booking['ticket_id'] == ticket_id:
            return booking
    return None
def sort_bookings(bookings, key):
    """
    Sorts bookings based on a specified key (either 'travel_date' or 'seat_number').

    Parameters:
    bookings (list): A list of booking records, where each record is a dictionary containing booking details.
    key (str): The key to sort by ('travel_date' or 'seat_number').
    Returns:
    list: A sorted list of booking records based on the specified key.
    """

    if key == 'travel_date':
        return sorted(bookings, key=lambda x: x['travel_date'])
    elif key == 'seat_number':
        return sorted(bookings, key=lambda x: x['seat_number'])
    else:
```

```python
    else:
        raise ValueError("Invalid key. Use 'travel_date' or 'seat_number'.")

# Example usage
if __name__ == "__main__":
    bookings = [
        {'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '12345', 'seat_number': 'A1', 'travel_date': '2024-08-01'},
        {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '12345', 'seat_number': 'A2', 'travel_date': '2024-08-01'},
        {'ticket_id': 'T003', 'passenger_name': 'Alice Johnson', 'train_number': '54321', 'seat_number': 'B1', 'travel_date': '2024-08-02'},
    ]

    # Search for a ticket
    ticket_id = 'T002'
    result = search_ticket(bookings, ticket_id)
    print(f"Search result for ticket ID {ticket_id}: {result}")

    # Sort bookings by travel date
    sorted_by_date = sort_bookings(bookings, key='travel_date')
    print("Bookings sorted by travel date:")
    for booking in sorted_by_date:
        print(booking)

    # Sort bookings by seat number
    sorted_by_seat = sort_bookings(bookings, key='seat_number')
    print("Bookings sorted by seat number:")
    for booking in sorted_by_seat:
        print(booking)
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_12.5.py
Search result for ticket ID T002: {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '12345', 'seat_number': 'A2', 'travel_date': '2024-08-01'}
Bookings sorted by travel date:
{'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '12345', 'seat_number': 'A1', 'travel_date': '2024-08-01'}
{'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '12345', 'seat_number': 'A2', 'travel_date': '2024-08-01'}
{'ticket_id': 'T003', 'passenger_name': 'Alice Johnson', 'train_number': '54321', 'seat_number': 'B1', 'travel_date': '2024-08-02'}
Bookings sorted by seat number:
{'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '12345', 'seat_number': 'A1', 'travel_date': '2024-08-01'}
{'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '12345', 'seat_number': 'A2', 'travel_date': '2024-08-01'}
{'ticket_id': 'T003', 'passenger_name': 'Alice Johnson', 'train_number': '54321', 'seat_number': 'B1', 'travel_date': '2024-08-02'}
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

```python
#A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:
#1. Search allocation details using student ID.
#2. Sort records based on room number or allocation date.
def search_allocation(allocations, student_id):
    """
    Searches for room allocation details by student ID.

    Parameters:
    allocations (list): A list of allocation records, where each record is a dictionary containing allocation details.
    student_id: the ID of the student to search for.

    Returns:
    dict: The allocation record if found, otherwise None.
    """

    for allocation in allocations:
        if allocation['student_id'] == student_id:
            return allocation
    return None
def sort_allocations(allocations, key):
    """
    Sorts room allocation records based on a specified key (either 'room_number' or 'allocation_date').
    Parameters:
    allocations (list): A list of allocation records, where each record is a dictionary containing allocation details.
    key (str): The key to sort by ('room_number' or 'allocation_date').
    Returns:
    list: A sorted list of allocation records based on the specified key.
    """
    if key == 'room_number':
        return sorted(allocations, key=lambda x: x['room_number'])
    elif key == 'allocation_date':
        return sorted(allocations, key=lambda x: x['allocation_date'])
    else:
        raise ValueError("Invalid key. Use 'room_number' or 'allocation_date'.")
```

```python
# Example usage
if __name__ == "__main__":
    allocations = [
        {'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-09-01'},
        {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-09-02'},
        {'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-09-01'},
    ]

    # Search for an allocation
    student_id = 'S002'
    result = search_allocation(allocations, student_id)
    print(f"Search result for student ID {student_id}: {result}")

    # Sort allocations by room number
    sorted_by_room = sort_allocations(allocations, key='room_number')
    print("Allocations sorted by room number:")
    for allocation in sorted_by_room:
        print(allocation)

    # Sort allocations by allocation date
    sorted_by_date = sort_allocations(allocations, key='allocation_date')
    print("Allocations sorted by allocation date:")
    for allocation in sorted_by_date:
        print(allocation)
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/lab assignment_12.5.py
Search result for student ID S002: {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-09-02'}
Allocations sorted by room number:
{'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-09-01'}
{'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-09-02'}
{'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-09-01'}
Allocations sorted by allocation date:
{'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-09-01'}
{'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-09-01'}
{'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-09-02'}
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #6: Online Movie Streaming Platform A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

• Recommend searching and sorting algorithms using AI.

• Justify the chosen algorithms.

• Implement Python functions.

```python
#Online Movie Streaming Platform A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:
#1. Search movies by movie ID.
#2. Sort movies based on rating or release year.
def search_movie(movies, movie_id):
    """
    Searches for a movie by its ID.

    Parameters:
    movies (list): A list of movie records, where each record is a dictionary containing movie details.
    movie_id: The ID of the movie to search for.

    Returns:
    dict: The movie record if found, otherwise None.

    """
    for movie in movies:
        if movie['movie_id'] == movie_id:
            return movie
    return None
def sort_movies(movies, key):
    """
    Sorts movies based on a specified key (either 'rating' or 'release_year').
    Parameters:
    movies (list): A list of movie records, where each record is a dictionary containing movie details.
    key (str): The key to sort by ('rating' or 'release_year').
    Returns:
    list: A sorted list of movie records based on the specified key.

    """
    if key == 'rating':
        return sorted(movies, key=lambda x: x['rating'], reverse=True)  # Sort by rating in descending order
    elif key == 'release_year':
        return sorted(movies, key=lambda x: x['release_year'])
    else:
        raise ValueError("Invalid key. Use 'rating' or 'release_year'.")
```

```python
# Example usage
if __name__ == "__main__":
    movies = [
        {'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010},
        {'movie_id': 'M002', 'title': 'The Shawshank Redemption', 'genre': 'Drama', 'rating': 9.3, 'release_year': 1994},
        {'movie_id': 'M003', 'title': 'The Godfather', 'genre': 'Crime', 'rating': 9.2, 'release_year': 1972},
    ]

    # Search for a movie
    movie_id = 'M002'
    result = search_movie(movies, movie_id)
    print(f"Search result for movie ID {movie_id}: {result}")

    # Sort movies by rating
    sorted_by_rating = sort_movies(movies, key='rating')
    print("Movies sorted by rating:")
    for movie in sorted_by_rating:
        print(movie)

    # Sort movies by release year
    sorted_by_year = sort_movies(movies, key='release_year')
    print("Movies sorted by release year:")
    for movie in sorted_by_year:
        print(movie)
```

```
PS C:\Users\gadah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gadah/OneDrive/Documents/AIAC/lab_assignment_12.5.py
Search result for movie ID M002: {'movie_id': 'M002', 'title': 'The Shawshank Redemption', 'genre': 'Drama', 'rating': 9.3, 'release_year': 1994}
Movies sorted by rating:
{'movie_id': 'M002', 'title': 'The Shawshank Redemption', 'genre': 'Drama', 'rating': 9.3, 'release_year': 1994}
{'movie_id': 'M003', 'title': 'The Godfather', 'genre': 'Crime', 'rating': 9.2, 'release_year': 1972}
{'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}
Movies sorted by release year:
{'movie_id': 'M003', 'title': 'The Godfather', 'genre': 'Crime', 'rating': 9.2, 'release_year': 1972}
{'movie_id': 'M002', 'title': 'The Shawshank Redemption', 'genre': 'Drama', 'rating': 9.3, 'release_year': 1994}
{'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}
PS C:\Users\gadah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #7: Smart Agriculture Crop Monitoring System An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

• Use AI-assisted reasoning to select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

```python
#Smart Agriculture Crop Monitoring System An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level,
#  temperature, and yield estimate. Farmers need to:
#1. Search crop details using crop ID.
#2. Sort crops based on moisture level or yield estimate.
def search_crop(crops, crop_id):
    """
    Searches for crop details by crop ID.

    Parameters:
    crops (list): A list of crop records, where each record is a dictionary containing crop details.
    crop_id: The ID of the crop to search for.

    Returns:
    dict: The crop record if found, otherwise None.

    """
    for crop in crops:
        if crop['crop_id'] == crop_id:
            return crop
    return None
def sort_crops(crops, key):
    """Sorts crops based on a specified key (either 'moisture_level' or 'yield_estimate').
    Parameters:
    crops (list): A list of crop records, where each record is a dictionary containing crop details.
    key (str): The key to sort by ('moisture_level' or 'yield_estimate').
    Returns:
    list: A sorted list of crop records based on the specified key.

    """
    if key == 'moisture_level':
        return sorted(crops, key=lambda x: x['soil_moisture_level'])
    elif key == 'yield_estimate':
        return sorted(crops, key=lambda x: x['yield_estimate'], reverse=True)  # Sort by yield estimate in descending order
    else:
        raise ValueError("Invalid key. Use 'moisture_level' or 'yield_estimate'.")
```

```python
# Example usage
if __name__ == "__main__":
    crops = [
        {'crop_id': 'C001', 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 500},
        {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 600},
        {'crop_id': 'C003', 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 700},
    ]

    # Search for a crop
    crop_id = 'C002'
    result = search_crop(crops, crop_id)
    print(f"Search result for crop ID {crop_id}: {result}")

    # Sort crops by moisture level
    sorted_by_moisture = sort_crops(crops, key='moisture_level')
    print("Crops sorted by moisture level:")
    for crop in sorted_by_moisture:
        print(crop)

    # Sort crops by yield estimate
    sorted_by_yield = sort_crops(crops, key='yield_estimate')
    print("Crops sorted by yield estimate:")
    for crop in sorted_by_yield:
        print(crop)
```

```
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_12.5.py
Search result for crop ID C002: {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 600}
Crops sorted by moisture level:
{'crop_id': 'C001', 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 500}
{'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 600}
{'crop_id': 'C003', 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 700}
Crops sorted by yield estimate:
{'crop_id': 'C003', 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 700}
{'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 600}
{'crop_id': 'C001', 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 500}
PS C:\Users\gudah\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #8: Airport Flight Management System An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

```python
#Airport Flight Management System An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must
#1. Search flight details using flight ID.
#2. Sort flights based on departure time or arrival time
def search_flight(flights, flight_id):
    """
    Searches for flight details by flight ID.

    Parameters:
    flights (list): A list of flight records, where each record is a dictionary containing flight details.
    flight_id: The ID of the flight to search for.

    Returns:
    dict: The flight record if found, otherwise None.

    """
    for flight in flights:
        if flight['flight_id'] == flight_id:
            return flight
    return None
def sort_flights(flights, key):
    """
    Sorts flights based on a specified key (either 'departure_time' or 'arrival_time').

    Parameters:
    flights (list): A list of flight records, where each record is a dictionary containing flight details.
    key (str): The key to sort by ('departure_time' or 'arrival_time').

    Returns:
    list: A sorted list of flight records based on the specified key.
    """
    if key == 'departure_time':
        return sorted(flights, key=lambda x: x['departure_time'])
    elif key == 'arrival_time':
        return sorted(flights, key=lambda x: x['arrival_time'])
    else:
        raise ValueError("Invalid key. Use 'departure_time' or 'arrival_time'.")
```

```python
#Example usage
if __name__ == "__main__":
    flights = [
        {'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'},
        {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'},
        {'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-10-01 07:30', 'arrival_time': '2024-10-01 09:30', 'status': 'On Time'},
    ]

    # Search for a flight
    flight_id = 'F002'
    result = search_flight(flights, flight_id)
    print(f"Search result for flight ID {flight_id}: {result}")

    # Sort flights by departure time
    sorted_by_departure = sort_flights(flights, key='departure_time')
    print("Flights sorted by departure time:")
    for flight in sorted_by_departure:
        print(flight)

    # Sort flights by arrival time
    sorted_by_arrival = sort_flights(flights, key='arrival_time')
    print("Flights sorted by arrival time:")
    for flight in sorted_by_arrival:
        print(flight)
```

```
PS C:\Users\gaddi\AppData\Local\Programs\Microsoft VS Code> & c:\python313\python.exe c:/Users/gaddi/OneDrive/Documents/AIAC/Lab_assignment_12.5.py
Search result for flight ID F002: {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}
Flights sorted by departure time:
{'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-10-01 07:30', 'arrival_time': '2024-10-01 09:30', 'status': 'On Time'}
{'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'}
{'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}
Flights sorted by arrival time:
{'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-10-01 07:30', 'arrival_time': '2024-10-01 09:30', 'status': 'On Time'}
{'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'}
{'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}
PS C:\Users\gaddi\AppData\Local\Programs\Microsoft VS Code>
```